Prompt: Módulo Cuenta Corriente — Empresas y Agencias

Contexto y problema

Hoy cuando se hace checkout de una reserva vinculada a una empresa o agencia con método "cuenta_corriente", el sistema registra un pago y cierra el folio — pero nadie sabe que esa empresa/agencia debe plata. No hay registro de esa deuda en ningún lado. Este prompt implementa el módulo completo de cuenta corriente.

PARTE 1 — Base de datos: nueva tabla `account_movements`

En `shared/schema.ts` **, agregar la tabla y sus tipos:**

```
export type AccountMovementType = "cargo" | "pago" | "nota_credito" | "ajuste";
export type AccountEntityType = "company" | "agency";

export const accountMovements = pgTable("account_movements", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  entityType: text("entity_type").$type<AccountEntityType>().notNull(),
  entityId: varchar("entity_id").notNull(),
  date: date("date").notNull(),
  type: text("type").$type<AccountMovementType>().notNull(),
  description: text("description").notNull(),
  amount: decimal("amount", { precision: 12, scale: 2 }).notNull(), // positivo =
  reservationId: varchar("reservation_id"),      // si vino de una reserva
  reservationCode: text("reservation_code"),     // desnormalizado para lectura r
  guestName: text("guest_name"),                 // desnormalizado
  reference: text("reference"),                  // nro transferencia, cheque, et
  createdBy: varchar("created_by"),
  createdAt: timestamp("created_at").notNull().defaultNow(),
});

export const insertAccountMovementSchema = createInsertSchema(accountMovements).o
export type InsertAccountMovement = z.infer<typeof insertAccountMovementSchema>;
export type AccountMovement = typeof accountMovements.$inferSelect;
```

También ampliar `BillingTarget` **:**

```
// Cambiar de:
export type BillingTarget = "guest" | "company";
// A:
export type BillingTarget = "guest" | "company" | "agency";
```

Agregar `accountMovements` al export de `schema.ts` junto a las otras tablas.

PARTE 2 — Migración de base de datos

En `server/migrate.ts` (o donde se aplican las migraciones), agregar:

```
await db.execute(sql`
  CREATE TABLE IF NOT EXISTS account_movements (
    id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),
    entity_type TEXT NOT NULL CHECK (entity_type IN ('company', 'agency')),
    entity_id VARCHAR NOT NULL,
    date DATE NOT NULL,
    type TEXT NOT NULL CHECK (type IN ('cargo', 'pago', 'nota_credito', 'ajuste')
    description TEXT NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    reservation_id VARCHAR,
    reservation_code TEXT,
    guest_name TEXT,
    reference TEXT,
    created_by VARCHAR,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
  )
`);
```

PARTE 3 — Storage: métodos nuevos

En `server/storage.ts`, agregar los siguientes métodos a la clase `DatabaseStorage` (y a la interfaz `IStorage`):

```
// Obtener movimientos de cuenta corriente de una entidad
async getAccountMovements(entityType: AccountEntityType, entityId: string): Promi
  return await db.select()
    .from(accountMovements)
    .where(
      and(
```

```
        eq(accountMovements.entityType, entityType),
        eq(accountMovements.entityId, entityId)
      )
    )
    .orderBy(desc(accountMovements.date), desc(accountMovements.createdAt));
}

// Calcular saldo actual (suma de todos los movimientos — positivo = deben, negat
async getAccountBalance(entityType: AccountEntityType, entityId: string): Promise
  const movements = await this.getAccountMovements(entityType, entityId);
  return movements.reduce((sum, m) => sum + parseFloat(m.amount), 0);
}

// Crear un movimiento
async createAccountMovement(data: InsertAccountMovement): Promise<AccountMovement
  const [created] = await db.insert(accountMovements).values(data as any).returni
  return created;
}

// Resumen de cuentas corrientes (para el dashboard de admin)
async getAccountSummary(): Promise<{
  companies: { id: string; name: string; balance: number; lastMovement: string |
  agencies: { id: string; name: string; balance: number; lastMovement: string | n
}> {
  const allMovements = await db.select().from(accountMovements);
  const allCompanies = await db.select().from(companies);
  const allAgencies = await db.select().from(agencies);

  const calcBalance = (entityType: string, entityId: string) =>
    allMovements
      .filter(m => m.entityType === entityType && m.entityId === entityId)
      .reduce((sum, m) => sum + parseFloat(m.amount), 0);

  const lastMovementDate = (entityType: string, entityId: string) => {
    const movements = allMovements
      .filter(m => m.entityType === entityType && m.entityId === entityId)
      .sort((a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).get
    return movements[0]?.date || null;
  };

  return {
    companies: allCompanies
      .filter(c => c.isActive === "true")
      .map(c => ({
        id: c.id,
        name: c.nombreFantasia || c.razonSocial,
        balance: calcBalance("company", c.id),
```

```
        lastMovement: lastMovementDate("company", c.id),
      }))
      .filter(c => c.balance !== 0), // solo las que tienen saldo
    agencies: allAgencies
      .filter(a => a.isActive === "true")
      .map(a => ({
        id: a.id,
        name: a.nombreFantasia || a.razonSocial,
        balance: calcBalance("agency", a.id),
        lastMovement: lastMovementDate("agency", a.id),
      }))
      .filter(a => a.balance !== 0),
  };
}
```

PARTE 4 — Endpoints API

En `server/routes.ts`, agregar los siguientes endpoints:

```
// GET movimientos + saldo de empresa
app.get("/api/companies/:id/account", async (req, res) => {
  try {
    const movements = await storage.getAccountMovements("company", req.params.id)
    const balance = await storage.getAccountBalance("company", req.params.id);
    res.json({ movements, balance });
  } catch (error) {
    res.status(500).json({ error: "Error fetching account" });
  }
});

// GET movimientos + saldo de agencia
app.get("/api/agencies/:id/account", async (req, res) => {
  try {
    const movements = await storage.getAccountMovements("agency", req.params.id);
    const balance = await storage.getAccountBalance("agency", req.params.id);
    res.json({ movements, balance });
  } catch (error) {
    res.status(500).json({ error: "Error fetching account" });
  }
});

// POST registrar pago recibido de empresa
app.post("/api/companies/:id/account/payment", async (req, res) => {
  try {
```

```
    const { amount, description, reference, date } = req.body;
    if (!amount || parseFloat(amount) <= 0) {
      return res.status(400).json({ error: "Monto inválido" });
    }
    const movement = await storage.createAccountMovement({
      entityType: "company",
      entityId: req.params.id,
      date: date || new Date().toISOString().split("T")[0],
      type: "pago",
      description: description || "Pago recibido",
      amount: (-parseFloat(amount)).toFixed(2), // negativo = acredita/reduce la
      reference: reference || null,
      createdBy: req.body.createdBy || null,
    });
    res.json(movement);
  } catch (error) {
    res.status(500).json({ error: "Error registering payment" });
  }
});

// POST registrar pago recibido de agencia
app.post("/api/agencies/:id/account/payment", async (req, res) => {
  try {
    const { amount, description, reference, date } = req.body;
    if (!amount || parseFloat(amount) <= 0) {
      return res.status(400).json({ error: "Monto inválido" });
    }
    const movement = await storage.createAccountMovement({
      entityType: "agency",
      entityId: req.params.id,
      date: date || new Date().toISOString().split("T")[0],
      type: "pago",
      description: description || "Pago recibido",
      amount: (-parseFloat(amount)).toFixed(2),
      reference: reference || null,
      createdBy: req.body.createdBy || null,
    });
    res.json(movement);
  } catch (error) {
    res.status(500).json({ error: "Error registering payment" });
  }
});

// GET resumen global de cuentas corrientes (para /admin)
app.get("/api/account-summary", async (req, res) => {
  try {
    const summary = await storage.getAccountSummary();
```

```
    res.json(summary);
  } catch (error) {
    res.status(500).json({ error: "Error fetching summary" });
  }
});
```

PARTE 5 — Trigger automático al hacer checkout con cuenta corriente

En el endpoint `POST /api/reservations/:id/checkout` **, buscar el bloque que actualiza el estado de la reserva y agregar el trigger ANTES del** `updateReservation` **:**

```
// En el checkout endpoint, luego de verificar que el balance es 0:

// Buscar si hay pagos en cuenta corriente de empresa o agencia
const reservationPayments = await db.select()
  .from(payments)
  .where(eq(payments.reservationId, req.params.id));

const ccPayment = reservationPayments.find(p => p.method === "cuenta_corriente");

if (ccPayment) {
  const reservation = await storage.getReservation(req.params.id);
  const guestName = reservation?.guest
    ? `${reservation.guest.firstName} ${reservation.guest.lastName}`
    : "Huésped";

  // Determinar si es empresa o agencia
  if (ccPayment.billingTarget === "company" && reservation?.companyId) {
    await storage.createAccountMovement({
      entityType: "company",
      entityId: reservation.companyId,
      date: new Date().toISOString().split("T")[0],
      type: "cargo",
      description: `Estadía ${reservation.reservationCode} — Hab. ${reservation.r
      amount: parseFloat(ccPayment.amount).toFixed(2), // positivo = deben
      reservationId: reservation.id,
      reservationCode: reservation.reservationCode,
      guestName,
    });
  } else if (ccPayment.billingTarget === "agency" && reservation?.agencyId) {
    await storage.createAccountMovement({
      entityType: "agency",
```

```
      entityId: reservation.agencyId,
      date: new Date().toISOString().split("T")[0],
      type: "cargo",
      description: `Estadía ${reservation.reservationCode} — Hab. ${reservation.r
      amount: parseFloat(ccPayment.amount).toFixed(2),
      reservationId: reservation.id,
      reservationCode: reservation.reservationCode,
      guestName,
    });
  }
}
```

PARTE 6 — UI: Tab "Cuenta Corriente" en página de Empresas

En `CompaniesPage` **, agregar:**

1. **Estado para la empresa seleccionada y el panel de cuenta corriente:**

```
const [viewingAccountCompany, setViewingAccountCompany] = useState<Company | null
const [registerPaymentOpen, setRegisterPaymentOpen] = useState(false);
const [paymentAmount, setPaymentAmount] = useState("");
const [paymentReference, setPaymentReference] = useState("");
const [paymentDescription, setPaymentDescription] = useState("Pago recibido");
const [paymentDate, setPaymentDate] = useState(new Date().toISOString().split("T"
```

2. **Query de cuenta corriente (solo cuando hay empresa seleccionada):**

```
const { data: accountData, refetch: refetchAccount } = useQuery<{
  movements: AccountMovement[];
  balance: number;
}>({
  queryKey: ["/api/companies", viewingAccountCompany?.id, "account"],
  queryFn: async () => {
    const res = await fetch(`/api/companies/${viewingAccountCompany!.id}/account`
    return res.json();
  },
  enabled: !!viewingAccountCompany,
});
```

3. **Mutation para registrar pago:**

```
const registerPaymentMutation = useMutation({
  mutationFn: async () => {
    return apiRequest("POST", `/api/companies/${viewingAccountCompany!.id}/accoun
      amount: paymentAmount,
      description: paymentDescription,
      reference: paymentReference || null,
      date: paymentDate,
    });
  },
  onSuccess: () => {
    refetchAccount();
    setRegisterPaymentOpen(false);
    setPaymentAmount("");
    setPaymentReference("");
    toast({ title: "Pago registrado en cuenta corriente" });
  },
  onError: () => {
    toast({ title: "Error al registrar pago", variant: "destructive" });
  },
});
```

4. **En la tabla de empresas, agregar un botón de cuenta corriente (con badge de saldo si corresponde) y al hacer clic abrir el panel lateral / sheet:**

En la columna Acciones de cada fila de empresa, agregar:

```
<Button
  variant="ghost"
  size="icon"
  onClick={() => setViewingAccountCompany(company)}
  title="Ver cuenta corriente"
  data-testid={`button-account-company-${company.id}`}
>
  <Receipt className="h-4 w-4 text-blue-600" />
</Button>
```

5. **Sheet de cuenta corriente (al final del JSX de CompaniesPage, antes del cierre del div):**

```
<Sheet open={!!viewingAccountCompany} onOpenChange={(open) => { if (!open) setVie
  <SheetContent className="w-full sm:max-w-2xl overflow-y-auto">
    <SheetHeader className="mb-4">
      <SheetTitle className="flex items-center gap-2">
        <Building2 className="h-5 w-5" />
        {viewingAccountCompany?.nombreFantasia || viewingAccountCompany?.razonSoc
```

```
  </SheetTitle>
  <SheetDescription>Cuenta corriente — movimientos y saldo</SheetDescription>
</SheetHeader>

{/* Saldo actual */}
<div className={`rounded-lg p-4 mb-4 flex items-center justify-between ${
  (accountData?.balance || 0) > 0
    ? "bg-red-50 border border-red-200 dark:bg-red-950/30 dark:border-red-800
    : "bg-green-50 border border-green-200 dark:bg-green-950/30 dark:border-g
}`}>
  <div>
    <p className="text-sm text-muted-foreground">Saldo actual</p>
    <p className={`text-3xl font-bold ${
      (accountData?.balance || 0) > 0 ? "text-red-600 dark:text-red-400" : "t
    }`}>
      ${Math.abs(accountData?.balance || 0).toFixed(2)}
    </p>
    <p className="text-xs text-muted-foreground mt-1">
      {(accountData?.balance || 0) > 0 ? "Saldo pendiente de cobro" : "Sin de
    </p>
  </div>
  <Button
    onClick={() => setRegisterPaymentOpen(true)}
    disabled={(accountData?.balance || 0) <= 0}
    data-testid="button-register-cc-payment"
  >
    <Plus className="h-4 w-4 mr-2" />
    Registrar pago
  </Button>
</div>

{/* Tabla de movimientos */}
{accountData?.movements && accountData.movements.length > 0 ? (
  <div className="border rounded-lg overflow-hidden">
    <Table>
      <TableHeader>
        <TableRow>
          <TableHead>Fecha</TableHead>
          <TableHead>Descripción</TableHead>
          <TableHead>Ref.</TableHead>
          <TableHead className="text-right">Monto</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        {accountData.movements.map((mov) => (
          <TableRow key={mov.id} data-testid={`movement-row-${mov.id}`}>
            <TableCell className="text-sm">{mov.date}</TableCell>
```

```
            <TableCell>
              <div>
                <p className="text-sm">{mov.description}</p>
                {mov.guestName && (
                  <p className="text-xs text-muted-foreground">{mov.guestName
                )}
              </div>
            </TableCell>
            <TableCell className="text-xs text-muted-foreground">{mov.referen
            <TableCell className={`text-right font-medium tabular-nums ${
              parseFloat(mov.amount) > 0 ? "text-red-600" : "text-green-600"
            }`}>
              {parseFloat(mov.amount) > 0 ? "+" : ""}${Math.abs(parseFloat(mo
              <span className="block text-xs font-normal text-muted-foregroun
                {parseFloat(mov.amount) > 0 ? "cargo" : "pago"}
              </span>
            </TableCell>
          </TableRow>
        ))}
      </TableBody>
    </Table>
  </div>
) : (
  <div className="text-center py-12 text-muted-foreground">
    <Receipt className="h-10 w-10 mx-auto mb-3 opacity-30" />
    <p className="text-sm">Sin movimientos en cuenta corriente</p>
  </div>
)}

{/* Dialog de registrar pago */}
<Dialog open={registerPaymentOpen} onOpenChange={setRegisterPaymentOpen}>
  <DialogContent>
    <DialogHeader>
      <DialogTitle>Registrar pago recibido</DialogTitle>
      <DialogDescription>
        {viewingAccountCompany?.nombreFantasia || viewingAccountCompany?.razo
        Saldo actual: ${(accountData?.balance || 0).toFixed(2)}
      </DialogDescription>
    </DialogHeader>
    <div className="grid gap-4 py-4">
      <div>
        <Label>Monto recibido</Label>
        <Input
          type="number"
          step="0.01"
          min="0.01"
          placeholder="0.00"
```

```
              value={paymentAmount}
              onChange={(e) => setPaymentAmount(e.target.value)}
              data-testid="input-cc-payment-amount"
            />
          </div>
          <div>
            <Label>Fecha</Label>
            <Input
              type="date"
              value={paymentDate}
              onChange={(e) => setPaymentDate(e.target.value)}
              data-testid="input-cc-payment-date"
            />
          </div>
          <div>
            <Label>Descripción</Label>
            <Input
              value={paymentDescription}
              onChange={(e) => setPaymentDescription(e.target.value)}
              placeholder="Ej: Pago por transferencia"
              data-testid="input-cc-payment-description"
            />
          </div>
          <div>
            <Label>Referencia (opcional)</Label>
            <Input
              value={paymentReference}
              onChange={(e) => setPaymentReference(e.target.value)}
              placeholder="Nro. transferencia, cheque, etc."
              data-testid="input-cc-payment-reference"
            />
          </div>
        </div>
        <DialogFooter>
          <Button variant="outline" onClick={() => setRegisterPaymentOpen(false)}
          <Button
            onClick={() => registerPaymentMutation.mutate()}
            disabled={!paymentAmount || parseFloat(paymentAmount) <= 0 || registe
            data-testid="button-confirm-cc-payment"
          >
            {registerPaymentMutation.isPending ? "Guardando..." : "Confirmar pago
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  </SheetContent>
</Sheet>
```

Agregar `Sheet, SheetContent, SheetHeader, SheetTitle, SheetDescription` **a los imports de shadcn/ui si no están. Agregar** `Receipt` **a los imports de lucide-react.**

PARTE 7 — UI: Tab "Cuenta Corriente" en página de Agencias

Repetir exactamente el mismo patrón de la Parte 6 pero en `AgenciesPage` **:**

- **Estado:** `viewingAccountAgency` **en lugar de** `viewingAccountCompany`
- **Query:** `/api/agencies/:id/account`
- **Mutation:** `POST /api/agencies/:id/account/payment`
- **Ícono del botón en tabla:** `<Plane className="h-4 w-4 text-indigo-600" />`
- **En la descripción del saldo de agencias, agregar info de comisión si** `commissionRate > 0` **:**

```
{parseFloat(viewingAccountAgency?.commissionRate || "0") > 0 && (
  <p className="text-xs text-muted-foreground mt-1">
    Comisión pactada: {viewingAccountAgency?.commissionRate}%
  </p>
)}
```

PARTE 8 — UI: Página `/admin` — cards de Cuenta Corriente con datos reales

En `admin.tsx` **, reemplazar las cards estáticas de "Cuenta Corriente Empresas" y "Cuenta Corriente Agencias" por versiones dinámicas:**

```
const { data: accountSummary } = useQuery<{
  companies: { id: string; name: string; balance: number }[];
  agencies: { id: string; name: string; balance: number }[];
}>({
  queryKey: ["/api/account-summary"],
});

const totalCompaniesDebt = accountSummary?.companies.reduce((sum, c) => sum + c.b
```

```
const totalAgenciesDebt = accountSummary?.agencies.reduce((sum, a) => sum + a.bal
```

En las cards correspondientes mostrar:

```
// Card Empresas
<p className="text-2xl font-bold text-red-600">${totalCompaniesDebt.toFixed(2)}</
<p className="text-xs text-muted-foreground">{accountSummary?.companies.length ||

// Card Agencias
<p className="text-2xl font-bold text-red-600">${totalAgenciesDebt.toFixed(2)}</p
<p className="text-xs text-muted-foreground">{accountSummary?.agencies.length ||
```

Cambiar el `status` de esas dos cards a `"available"` y el `href` a `/companies` y `/agencies` respectivamente (por ahora navegan ahí donde está el panel de cuenta corriente).

Resumen de archivos modificados

Archivo	Cambio
`shared/schema.ts`	**Nueva tabla `accountMovements`, ampliar `BillingTarget`**
`server/migrate.ts`	**CREATE TABLE account_movements**
`server/storage.ts`	**4 métodos nuevos: getAccountMovements, getAccountBalance, createAccountMovement, getAccountSummary**
`server/routes.ts`	**5 endpoints nuevos + trigger en checkout**
`client/src/pages/companies.tsx`	**Sheet de cuenta corriente + botón en tabla**
`client/src/pages/agencies.tsx`	**Sheet de cuenta corriente + botón en tabla**
`client/src/pages/admin.tsx`	**Cards dinámicas con datos reales de deuda**